

Informe de pruebas

Alke Wallet · Módulo 7 · Django

Full Stack Python Trainee · Talento Digital / Alkemy

Autor: Javier Muñoz Llanos

Octubre de 2026

1. Objetivo y alcance

Este informe documenta las pruebas automatizadas de Alke Wallet y sus resultados. El objetivo es demostrar que las reglas de negocio, los formularios, las pantallas y la seguridad funcionan como se espera, y que las pruebas fallan cuando una regla se rompe.

Alcance:

- Modelos, restricciones y validaciones.
- Servicios con lógica de negocio (movimientos atómicos y saldos).
- Consultas con ORM, agregaciones y SQL propio.
- Formularios y vistas basadas en clases (CRUD completo).
- Seguridad: autenticación, roles, alcance por cliente y CSRF.
- Comprobación de salud del servicio y comando de despliegue crear_superusuario

Fuera del alcance: pruebas de carga y pruebas automáticas de concurrencia (ver sección 8). Pruebas automáticas de interfaz y del despliegue en línea (se verificaron manualmente; ver documento técnico, etapas 9 y 10)

2. Entorno y ejecución

Elemento	Valor
Lenguaje y framework	Python 3.14 · Django 6.1
Base de datos de pruebas	SQLite (Django crea una base temporal y la elimina al terminar)
Herramienta	django.test.TestCase (cada prueba se deshace al terminar) y Client
Ubicación	gestion/tests/ (un archivo por bloque)
Rama	feature/tests (239 pruebas) y feature/espera y feature/render (6 pruebas adicionales)

Comando para ejecutar todo:

```
python manage.py test gestion
```

Comando para ejecutar un bloque con detalle, por ejemplo seguridad:

```
python manage.py test gestion.tests.test_seguridad -v 2
```

Comprobación manual adicional de accesos (no deja datos en la base):

```
python manage.py verificar_accesos
```

3. Resumen de resultados

Archivo	Bloque	Pruebas	Correctas	Fallidas
test_humo.py	Prueba de humo	5	5	0
test_modelos.py	Modelos y restricciones	42	42	0
test_servicios.py	Servicios (lógica de negocio)	21	21	0
test_consultas.py	Consultas ORM, agregaciones y SQL	31	31	0
test_formularios.py	Formularios	41	41	0
test_vistas.py	Vistas como personal	49	49	0

Archivo	Bloque	Pruebas	Correctas	Fallidas
test_seguridad.py	Seguridad	50	50	0
test_salud.py	Salud del servicio	2	2	0
Test_comandos.py	Comando crear_superusuario	4	4	0
TOTAL		245	245	0

Resultado global: 245 pruebas ejecutadas, 245 correctas, 0 fallidas. El comando verificar_accesos obtuvo 55 de 55 comprobaciones correctas.

4. Casos de prueba

Cada fila agrupa las pruebas que revisan una misma regla. La columna "Obtenido" indica cuántas pruebas del grupo salieron correctas. El detalle de cada prueba individual se ve ejecutando el comando con -v 2.

4.1 Prueba de humo

ID	Qué se prueba	Cómo	Resultado esperado	Obtenido
HUM-01	Prueba de humo: el entorno de pruebas funciona	Base vacía, aislamiento entre pruebas, login público y pantalla privada que redirige	Todas las comprobaciones correctas	Correcto (5/5)

4.2 Modelos

ID	Qué se prueba	Cómo	Resultado esperado	Obtenido
MOD-01	Moneda: código único y texto al imprimir	Crear monedas repetidas y comparar __str__	Error de integridad al repetir; texto "CLP"	Correcto (2/2)
MOD-02	Cliente: relación 1:1 con usuario, correo único, teléfono opcional	Crear clientes con datos repetidos o vacíos	Se rechazan duplicados; teléfono admite NULL	Correcto (7/7)
MOD-03	Contacto: no agendarse a sí mismo ni repetir ficha	Crear fichas inválidas (clean y restricciones de BD)	ValidationError / IntegrityError	Correcto (6/6)
MOD-04	Cuenta: número único, saldo = entradas – salidas, protección al borrar	Crear cuentas, movimientos y borrar	Saldo correcto; ProtectedError con movimientos	Correcto (7/7)
MOD-05	Transacción: reglas por tipo, monto positivo, orden y texto	clean() con combinaciones válidas e inválidas; restricción de BD	Solo se aceptan las combinaciones válidas	Correcto (20/20)

4.3 Servicios

ID	Qué se prueba	Cómo	Resultado esperado	Obtenido
SER-01	registrar_transaccion: depósito, retiro y transferencia	Llamar al servicio con saldos distintos	Movimiento guardado; saldo insuficiente rechazado	Correcto (14/14)
SER-02	Bloqueo de la cuenta origen (select_for_update) y atomicidad	Espiar el bloqueo con mock y forzar errores	Se bloquea la cuenta; sin movimientos a medias	Incluido en SER-01 y SER-04
SER-03	siguiente_numero_cuenta	Pedir número con y sin cuentas existentes	Siguiente número libre de 4 dígitos	Correcto (4/4)

ID	Qué se prueba	Cómo	Resultado esperado	Obtenido
SER-04	crear_cliente_con_cuenta	Crear y forzar un fallo con mock	Cliente y cuenta juntos o ninguno	Correcto (3/3)

4.4 Consultas

ID	Qué se prueba	Cómo	Resultado esperado	Obtenido
CON-01	Filtros del ORM (teléfono, sin contactos, recientes, de un cliente)	Conjunto de datos fijo (5 clientes, 7 cuentas)	Resultados exactos	Correcto (8/8)
CON-02	Agregaciones (resumen por tipo, saldos por cliente, cuentas con saldo)	Comparar contra cifras calculadas a mano	Totales idénticos	Correcto (13/13)
CON-03	Movimientos por mes	Fechas fijas en meses distintos	Conteo por mes correcto	Correcto (3/3)
CON-04	Consultas SQL propias (raw y cursor)	Comparar con las consultas del ORM y probar texto de búsqueda	Mismos resultados que el ORM	Correcto (7/7)

4.5 Formularios

ID	Qué se prueba	Cómo	Resultado esperado	Obtenido
FOR-01	ClienteForm y ClienteEdicionForm	Datos válidos, usuario ocupado, correo repetido, nombre vacío	Validan o rechazan según la regla	Correcto (8/8)
FOR-02	CuentaForm y CuentaEdicionForm	Número repetido y campos editables	Mensaje "Ya existe una cuenta con ese número."	Correcto (4/4)
FOR-03	TransaccionForm (personal)	Opciones, etiquetas, cuentas inactivas, monto 0, tipo vacío	Solo cuentas activas; errores en campos inválidos	Correcto (9/9)
FOR-04	FiltroTransaccionForm	Vacío, válido, fecha mal escrita, tipo desconocido	Vacío válido; datos malos inválidos	Correcto (4/4)
FOR-05	ContactoForm	Excluir al dueño y a los ya agendados; guardar	Solo clientes agendables	Correcto (6/6)
FOR-06	RegistroForm	Alta completa, claves distintas o débiles, correo repetido con mayúsculas, moneda vacía, falla a medias	Crea usuario + cliente + cuenta; si falla no queda nada	Correcto (10/10)

4.6 Vistas

ID	Qué se prueba	Cómo	Resultado esperado	Obtenido
VIS-01	Inicio	Entrar como personal y revisar totales	Totales correctos; máximo 5 movimientos	Correcto (2/2)
VIS-02	Clientes: lista, crear, detalle, editar, eliminar	GET/POST como personal; borrar con y sin movimientos	Avisos exactos; cliente con movimientos queda protegido	Correcto (10/10)
VIS-03	Cuentas: lista, crear, detalle, editar, eliminar	GET/POST; ?cliente=; 17 movimientos; borrar con y sin movimientos	Número sugerido; máximo 15 movimientos; protección al borrar	Correcto (10/10)

ID	Qué se prueba	Cómo	Resultado esperado	Obtenido
VIS-04	Movimientos: lista, filtros, paginación, registrar, detalle	12 movimientos, filtros tipo/cuenta/fechas, POST de depósito, retiro sin saldo y transferencia	10 por página; filtros correctos; sin saldo no se guarda	Correcto (12/12)
VIS-05	Contactos: agregar y quitar	POST como personal	Aviso exacto; al quitar la ficha el cliente sigue existiendo	Correcto (4/4)
VIS-06	Reporte	Dos monedas con movimientos y base sin datos	Una sección por moneda sin mezclar saldos; no falla vacío	Correcto (3/3)
VIS-07	Registro público	Alta válida e inválida; usuario ya con sesión	Inicia sesión y va al inicio; con error no crea nada	Correcto (4/4)
VIS-08	Perfil y cambio de contraseña	Clave correcta, incorrecta y sesión posterior	Aviso de éxito; la sesión sigue abierta	Correcto (4/4)

4.7 Seguridad

ID	Qué se prueba	Cómo	Resultado esperado	Obtenido
SEG-01	Sin sesión	Pedir 18 pantallas privadas y hacer POST anónimos	Redirige al acceso; no cambia datos; login y registro públicos	Correcto (4/4)
SEG-02	Roles	Cliente, usuario sin cliente y personal contra cada pantalla	403 según rol; el personal abre todo; plantilla 403 propia	Correcto (8/8)
SEG-03	Alcance por cliente	Ana intenta abrir, editar o borrar registros de Luis	404 y sin cambios; listados solo con lo propio	Correcto (11/11)
SEG-04	Reglas de operación del cliente	POST con cuentas ajenas, contactos y no contactos	Retiro y origen solo propios; depósito solo propio; transferir solo a contactos	Correcto (10/10)
SEG-05	CSRF	Client con enforce_csrf_checks: sin token, token falso y válido	403 sin token o falso; funciona con token válido	Correcto (7/7)
SEG-06	Acceso y sesión	Login, usuario desactivado, next externo, logout, registro con is_staff, cabecera X-Frame-Options	Sin redirección externa; logout solo POST; registro nunca crea personal	Correcto (10/10)

4.8 Salud del servicio

ID	Qué se prueba	Cómo	Resultado esperado	Obtenido
SAL-01	/salud/ responde sin iniciar sesión	Petición GET anónima a /salud/	Código 200 y cuerpo {"estado": "ok"}	Correcto (1/1)
SAL-2	Permite que la lea una página de otro dominio	Revisar los encabezados de la respuesta de /salud/	Access-Control-Allow-Origin: *; sin él, la página de espera de GitHub Pages no podría leerla	Correcto (1/1)

4.9 Comandos

ID	Qué se prueba	Cómo	Resultado esperado	Obtenido
CMD-01	crear_superusuario crea el administrador con las variables de entorno	Ejecutar el comando con DJANGO_SUPERUSER_USERNAME, _EMAIL y _PASSWORD definidas	Se crea un usuario superusuario y del personal, con la clave indicada	Correcto (1/1)
CMD-02	No hace nada si faltan las variables	Ejecutar el comando sin ninguna variable	No se crea ningún usuario y el comando no falla	Correcto (1/1)
CMD-03	Repetirlo no cambia la clave	Ejecutarlo dos veces; la segunda con otra clave	El usuario sigue existiendo y conserva la clave original	Correcto (1/1)
CMD-04	No muestra la clave en pantalla	Capturar la salida del comando	La salida no contiene la clave	Correcto (1/1)

5. Verificación de que las pruebas detectan fallos

Una prueba que siempre pasa no demuestra nada. Por eso se rompió a propósito cada regla de seguridad y de despliegue, se ejecutaron las pruebas de seguridad y después se restauró el código original. En todos los casos al menos una prueba falló.

Cambio introducido a propósito	Pruebas que fallaron
Quitar el filtro por cliente (alcance)	8
Quitar la regla del depósito propio	1
Dejar abiertas las pantallas del personal	16
Quitar el middleware CSRF	10
Quitar el login obligatorio	3
Dejar abiertas las pantallas de cliente	8
Cuentas de origen sin filtro	4
Destino sin limitar a contactos	3
Movimientos visibles sin filtro	5
Quitar el encabezado CORS de /salud/	1
Crear el superusuario aunque falten las variables	1
Pisar al superusuario si ya existe	1
Escribir la clave en pantalla	1

Limitación detectada: en SQLite la búsqueda con LIKE ya ignora mayúsculas, por lo que quitar LOWER() de la búsqueda SQL propia no se detecta con esta base. En PostgreSQL ese cambio sí tendría efecto.

6. Comprobación manual: verificar accesos

El comando verificar_accesos crea datos temporales (personal, dos clientes y un usuario sin cliente), recorre las pantallas con cada rol, revisa que los formularios exijan el token CSRF y deshace todo al final. Obtuvo 55 de 55 comprobaciones correctas, con el mismo comportamiento que las pruebas formales de seguridad.

7. Evidencias (capturas)

Cada recuadro corresponde a una captura de la terminal con el comando indicado. Reemplazar el recuadro por la imagen.

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\Proyecto\alke_wallet> git checkout -b feature/tests
Switched to a new branch 'feature/tests'
```

Captura 129: Rama feature/tests creada

```
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> python manage.py test
Found 5 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 5 tests in 0.042s

OK
Destroying test database for alias 'default'...
```

Captura 130: Prueba de humo

```
-----
Ran 42 tests in 0.323s

OK
Destroying test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Todas las pruebas de la app
>> python manage.py test gestion
Found 47 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 47 tests in 0.809s

OK
Destroying test database for alias 'default'...
```

Captura 131 y 132: Pruebas de modelos

```
-----
Ran 21 tests in 0.279s

OK
Destroying test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Todas las pruebas de la app
>> python manage.py test gestion
Found 68 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 68 tests in 0.815s

OK
Destroying test database for alias 'default'...
```

Captura 133 y 134: Pruebas de servicios

```

-----
Ran 31 tests in 0.397s

OK
Destroying test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...
(venv) PS D:\VsCode_Externo\Bootcamp_Python\M7_Acceso_datos_app_Django\Proyecto\alke_wallet> # Todas las pruebas de la app
>> python manage.py test gestion
Found 99 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
-----
Ran 99 tests in 0.892s

OK
Destroying test database for alias 'default'...

```

Captura 135 y 136: Pruebas de consultas

```

-----
Ran 41 tests in 1.572s

OK
Destroying test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...

```

Captura 137: Pruebas de formularios (41 pruebas)

```

-----
Ran 49 tests in 7.087s

OK
Destroying test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...

```

Captura 138: Pruebas de vistas (49 pruebas)

```

-----
Ran 189 tests in 8.123s

OK
Destroying test database for alias 'default'...

```

Captura 139: Suite completa tras 8.4 (189 pruebas)

```

-----
Ran 50 tests in 5.730s

OK
Destroying test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...

```

Captura 140: Pruebas de seguridad (50 pruebas)

```

-----
Ran 239 tests in 13.094s

OK
Destroying test database for alias 'default'...

```

Captura 141: Suite completa de la etapa 8 (239 pruebas)

```

OK      cliente: /contactos/7/eliminar/ (ajeno)      404
OK      sin cliente: /perfil/                        200
OK      sin cliente: / (inicio con aviso)            200
OK      sin cliente: /cuentas/                      403
OK      sin cliente: /transacciones/                403
OK      sin cliente: /clientes/                     403
OK      sin cliente: /transacciones/nueva/          403
OK      CSRF: login sin token                       403
OK      CSRF: registro sin token                    403
OK      CSRF: cerrar sesión sin token               403
OK      CSRF: cambiar contraseña sin token          403
OK      CSRF: nueva transacción sin token            403
OK      CSRF: editar mis datos sin token             403
OK      logout con GET no permitido                  405
OK      CSRF: login con token (clave mala) se procesa 200

55 de 55 pruebas correctas. Los datos de prueba se deshicieron.

```

Captura 142: Comando verificar_accesos (55 de 55)

```

Forbidden (CSRF cookie not set.): /transacciones/nueva/
Forbidden (CSRF cookie not set.): /clientes/1/contactos/nuevo/
Forbidden (CSRF cookie not set.): /perfil/clave/
Forbidden (CSRF cookie not set.): /clientes/1/eliminar/
Forbidden (CSRF cookie not set.): /registro/
Forbidden (CSRF cookie not set.): /acceso/login/
Forbidden (CSRF token from POST incorrect.): /transacciones/nueva/
.....
-----
Ran 245 tests in 22.589s
OK
Destroying test database for alias 'default'...

```

Captura 143: 245 pruebas OK

8. Conclusiones y limitaciones

- Las 245 pruebas pasan y cubren modelos, servicios, consultas, formularios, vistas y seguridad.
- Las pruebas se ejecutaron en SQLite. Para PostgreSQL basta cambiar DB_ENGINE, pero el usuario de la base necesita permiso CREATEDB para crear la base de pruebas.
- El bloqueo de filas con select_for_update solo tiene efecto real en PostgreSQL. SQLite lo ignora, por lo que la concurrencia no se prueba de forma automática.
- Mejora futura: medir cobertura de código (coverage.py) y agregar pruebas de interfaz con un navegador automatizado.
- El diseño visual y el despliegue en Render se verificaron manualmente y con comprobaciones externas, no con pruebas automáticas